ΜΑΘΗΜΑ 1 ΑΠΟ 15 · ΥΠΟΕΝΟΤΗΤΑ 2.3.Γ.1

Γνωριμία με την Python

Από το πρώτο print() μέχρι ένα πρόγραμμα που δίνει ρέστα

Θεωρία + 8 ασκήσεις με λύσεις Διάρκεια: 4 ώρες (3η–6η) 26 διαφάνειες

Προγραμματισμός Εφαρμογών Διαδικτύου & Παιχνιδιών (Python, Pygame) · Γ΄ εξάμηνο

Τι θα ξέρεις στο τέλος

1. Να εγκαθιστάς την Python και να τρέχεις κώδικα **στο REPL** και **από αρχείο .py**.
2. Να γράφεις σωστά δομημένο κώδικα με **εσοχές**, χωρίς δηλώσεις τύπων και ερωτηματικά.
3. Να χρησιμοποιείς μεταβλητές και τους 4 βασικούς τύπους: **int, float, str, bool**.
4. Να ξεχωρίζεις τους τελεστές **/**, **//** και **%** — και να ξέρεις πότε χρειάζεσαι τον καθένα.
5. Να διαβάζεις από τον χρήστη με **input()** και να μετατρέπεις σωστά τον τύπο.
6. Να γράφεις μόνος σου ένα πλήρες πρόγραμμα: **ΔΜΣ, θερμοκρασία, ρέστα σε κέρματα**.

Πώς είναι οργανωμένο το 4ωρο

Ώρες 1–2: θεωρία με live coding — γράφουμε μαζί κάθε παράδειγμα. **Ώρες 3–4:** εργαστήριο — οι ασκήσεις πάνε από τις πολύ εύκολες στο τελικό πρόγραμμα «ρέστα σε κέρματα».

ΣΗΜΕΊΩΣΗ Το μόνο που χρειάζεσαι σήμερα είναι ένας υπολογιστής με Python. Κάθε παράδειγμα το γράφουμε μαζί, βήμα-βήμα.

Μία γλώσσα, πολλές δουλειές

Η Python είναι σταθερά στις πιο δημοφιλείς γλώσσες. Θα τη χρησιμοποιήσουμε και για web και για παιχνίδια.

Web

Back-end ιστοσελίδων με **Flask** και **Django**. Το «Διαδικτύου» στον τίτλο του μαθήματος.

Δεδομένα & AI

Ανάλυση με **pandas**, γραφήματα με **Matplotlib**, μηχανική μάθηση. Εδώ κυριαρχεί.

Παιχνίδια

Με τη βιβλιοθήκη **pygame** φτιάχνουμε 2D παιχνίδια. Είναι ο στόχος του εξαμήνου.

Αυτοματισμοί

Μικρά scripts που μετονομάζουν αρχεία, διαβάζουν Excel, στέλνουν email.

```
>>> import this
The Zen of Python, by Tim Peters
Beautiful is better than ugly.
Simple is better than complex.
Readability counts.
```

Η φιλοσοφία της

Γράψε **import this** στο REPL. Η Python θέλει κώδικα που **διαβάζεται σαν κείμενο**.
Γι' αυτό έχει λίγα σύμβολα και πολλές καθαρές λέξεις.

ΚΑΛΌ ΝΑ ΞΈΡΕΙΣ Το όνομα δεν βγαίνει από το φίδι αλλά από τους Monty Python. Ο δημιουργός, Guido van Rossum, ξεκίνησε τη γλώσσα το 1989.

Στήνουμε το εργαστήριο

Τέσσερα βήματα, 10 λεπτά. Αν η Python υπάρχει ήδη στο PC, ξεκίνα από το βήμα 3.

1. Κατέβασε την Python 3 από το **python.org/downloads** (Windows installer).
2. Στην πρώτη οθόνη τσέκαρε **«Add python.exe to PATH»** και πάτα Install Now.
3. Άνοιξε τερματικό (cmd) και γράψε **python --version**. Πρέπει να δεις τον αριθμό έκδοσης.
4. Γράψε κώδικα στο **IDLE** (έρχεται μαζί) ή στο **VS Code** με το extension «Python».

```
$ python --version
Python 3.12.6
$ python
>>> print("Γεια!")
Γεια!
```

Αν δεν δουλεύει το «python»

Συνήθως ξεχάστηκε το **PATH** στο βήμα 2. Ξανατρέξε τον installer → Modify, ή δοκίμασε την εντολή **py**.
Για έξοδο από το REPL: **exit()**.

ΠΡΟΣΟΧΉ Θέλουμε **Python 3**. Κώδικας Python 2 (π.χ. print χωρίς παρενθέσεις) που θα βρεις στο internet δεν τρέχει.

REPL ή αρχείο .py;

1 · REPL (ΔΙΑΔΡΑΣΤΙΚΆ)

```
>>> 2 + 3
5
>>> 10 / 4
2.5
>>> "Py" * 3
'PyPyPy'
```

2 · ΑΡΧΕΊΟ hello.py

```
# hello.py
name = "Γλυφάδα"
print("Γεια σου,", name)
print(2 + 3)

# τρέχει με:  python hello.py
```

Πότε το REPL

Για **δοκιμές μιας γραμμής**: «τι βγάζει το 7 // 2;». Απαντά αμέσως, αλλά ό,τι γράψεις **χάνεται** όταν κλείσει.

Πότε αρχείο

Για **κανονικά προγράμματα**. Σώζεται, ξανατρέχει, το διορθώνεις. Στο IDLE: File → New File, μετά **F5**.

Η διαφορά που μπερδεύει

Στο REPL το **2 + 3** εμφανίζει 5 μόνο του. Σε αρχείο **δεν εμφανίζεται τίποτα** — χρειάζεται **print(2 + 3)**.

ΣΎΜΒΑΣΗ Στις διαφάνειες, το **>>>** σημαίνει REPL. Οι γραμμές χωρίς >>> από κάτω είναι η απάντηση της Python.

Το ίδιο πρόγραμμα σε C και σε Python

C (Α΄ ΕΞΆΜΗΝΟ)

```c
#include <stdio.h>

int main(void) {
    int x = 5;
    printf("x = %d\n", x);
    return 0;
}
```

PYTHON

```python
x = 5
print("x =", x)
```

ΣΤΗ C	ΣΤΗΝ PYTHON	ΓΙΑΤΊ
`#include, main()`	δεν χρειάζονται	το αρχείο τρέχει από πάνω προς τα κάτω
`int x = 5;`	`x = 5`	ο τύπος βγαίνει **από την τιμή**
`;` στο τέλος	τίποτα	κάθε γραμμή είναι μία εντολή
`printf("%d", x)`	`print(x)`	η print ξέρει μόνη της τι τυπώνει
μεταγλώττιση → .exe	εκτέλεση απευθείας	η Python είναι **διερμηνευόμενη**

ΚΛΕΙΔΊ Δύο γραμμές αντί για επτά. Η λογική είναι ίδια — αλλάζει μόνο το πόσα πρέπει να γράψεις για να την πεις.

Τα κενά στην αρχή έχουν σημασία

Στη C τα μπλοκ τα κλείνουν οι { }. Στην Python τα κλείνει η εσοχή — 4 κενά.

C

```c
if (x > 0) {
    printf("θετικός");
    y = 1;
}
printf("τέλος");
```

PYTHON

```python
if x > 0:
    print("θετικός")
    y = 1
print("τέλος")
```

```
print("A")
  print("B")      # 2 κενά χωρίς λόγο
IndentationError: unexpected indent
```

Το πιο συχνό πρώτο λάθος

Ένα κενό στην αρχή γραμμής, εκεί που δεν ανοίγει μπλοκ, και το πρόγραμμα **δεν τρέχει καθόλου**.

Κανόνας του εργαστηρίου

Πάντα **4 κενά** ανά επίπεδο. Το VS Code και το IDLE τα βάζουν μόνα τους όταν πατάς Enter μετά από **:**. Το if το βλέπουμε αναλυτικά στο Μάθημα 2.

ΓΙΑΤΊ ΈΤΣΙ Ο κώδικας αναγκαστικά **φαίνεται** όπως **δουλεύει**. Δεν γίνεται να είναι σωστά στοιχισμένος αλλά λάθος δομημένος.

Η print() σε τέσσερις μορφές

ΈΝΑ ΠΡΆΓΜΑ

```
print("Καλημέρα")
Καλημέρα
```

ΠΟΛΛΆ, ΜΕ ΚΌΜΜΑ

```
print("Έτος:", 2026, "!")
Έτος: 2026 !
```

ΆΛΛΟ ΔΙΑΧΩΡΙΣΤΙΚΌ: SEP

```
print(5, 10, 2026, sep="/")
5/10/2026
```

ΧΩΡΊΣ ΑΛΛΑΓΉ ΓΡΑΜΜΉΣ: END

```
print("A", end=" ")
print("B")
A B
```

Σχόλια με #

Ό,τι ακολουθεί το # το αγνοεί η Python — είναι σημειώσεις για ανθρώπους. Βάζε σχόλιο όταν το **γιατί** δεν φαίνεται από τον κώδικα.

ΠΡΟΣΈΞΕ Με κόμμα η print βάζει **μόνη της ένα κενό** ανάμεσα. Γι' αυτό γράφουμε "Έτος:" και όχι "Έτος: ".

Όνομα που δείχνει σε μια τιμή

```
age = 19
price = 4.50
name = "Μαρία"

age = age + 1       # τώρα 20
age = "είκοσι"      # επιτρέπεται!
print(age)
είκοσι
```

ΌΝΟΜΑ	ΙΣΧΎΕΙ;	ΓΙΑΤΊ
`total_price`	ναι	snake_case: το στυλ της Python
`2nd_try`	όχι	δεν ξεκινά με ψηφίο
`my-var`	όχι	το - είναι αφαίρεση
`print`	ναι, αλλά ΜΗ	χαλάει την print()
`Age / age`	και τα δύο	είναι **διαφορετικές** μεταβλητές

Καμία δήλωση

Η μεταβλητή **δημιουργείται** την πρώτη φορά που της δίνεις τιμή με =. Δεν γράφεις int ή float πριν.

Δυναμικός τύπος

Η ίδια μεταβλητή μπορεί να πάρει μετά **άλλου τύπου** τιμή. Η Python δεν διαμαρτύρεται — γι' αυτό προσέχουμε εμείς.

ΣΥΜΒΟΥΛΉ Ονόματα που λένε τι κρατάνε: **weight**, **height**, **bmi** — όχι **a**, **b**, **c**. Σε δύο εβδομάδες θα ευχαριστείς τον εαυτό σου.

Οι τέσσερις βασικοί τύποι

Κάθε τιμή έχει τύπο. Τον βλέπεις με τη συνάρτηση type().

ΤΎΠΟΣ	ΤΙ ΚΡΑΤΆΕΙ	ΠΑΡΑΔΕΊΓΜΑΤΑ	ΠΡΟΣΟΧΉ
`int`	ακέραιους	`7, 0, -25, 10**20`	**κανένα όριο** μεγέθους
`float`	δεκαδικούς	`3.14, 2.0, -0.5`	τελεία, όχι κόμμα
`str`	κείμενο	`"Μαρία", '42'`	"42" **δεν** είναι αριθμός
`bool`	αληθές / ψευδές	`True, False`	κεφαλαίο το πρώτο γράμμα

```
>>> type(7)
<class 'int'>
>>> type(7.0)
<class 'float'>
>>> type("7")
<class 'str'>
```

Τρεις διαφορετικοί «επτά»

7, **7.0** και **"7"** μοιάζουν, αλλά είναι τρία διαφορετικά πράγματα για την Python.
Το **"7" + 1** βγάζει σφάλμα. Το **7 + 1.0** βγάζει **8.0** (float).

ΜΝΗΜΟΝΙΚΌ Εισαγωγικά = κείμενο. Τελεία = float. Τίποτα από τα δύο = int.

Τι τύπου είναι;

ΕΚΦΏΝΗΣΗ

Χωρίς υπολογιστή: γράψε τι θα επιστρέψει η type() για καθεμία τιμή. Μετά έλεγξέ το στο REPL.

ΛΎΣΗ

ΤΙΜΉ	ΤΎΠΟΣ	ΓΙΑΤΊ
`42`	int	ακέραιος χωρίς τελεία
`42.0`	float	έχει τελεία
`"42"`	str	έχει εισαγωγικά
`False`	bool	λέξη-κλειδί, κεφαλαίο F
`7 / 7`	**float**	η / δίνει **πάντα** float → 1.0
`"True"`	**str**	εισαγωγικά νικούν

```
type(42)
type(42.0)
type("42")
type(False)
type(7 / 7)
type("True")
```

Οι δύο παγίδες

7 / 7 δεν είναι 1 αλλά **1.0** — η / δίνει **πάντα** float.
"True" είναι κείμενο που απλώς γράφει True.

ΣΤΌΧΟΣ Να «βλέπεις» τον τύπο με το μάτι πριν τρέξεις τον κώδικα. Θα το χρειαστείς στην input() σε λίγο.

Αριθμητικές πράξεις

ΤΕΛ.	ΠΡΆΞΗ	ΠΑΡΆΔΕΙΓΜΑ	ΑΠΟΤ.
+	πρόσθεση	`7 + 2`	9
-	αφαίρεση	`7 - 2`	5
*	πολλαπλασιασμός	`7 * 2`	14
/	διαίρεση	`7 / 2`	3.5
//	ακέραιο πηλίκο	`7 // 2`	3
%	υπόλοιπο	`7 % 2`	1
**	δύναμη	`7 ** 2`	49

Η μεγάλη αλλαγή

Η **/** δίνει **πάντα δεκαδικό**: 6 / 3 = **2.0** (στη C το 7/2 έδινε 3). Για ακέραιο πηλίκο υπάρχει ξεχωριστός τελεστής: **//**.

Προτεραιότητα

Πρώτα **, μετά * / // %, μετά + -. Όταν διστάζεις, βάλε **παρενθέσεις**.

ΔΕΝ ΥΠΆΡΧΕΙ Η Python **δεν έχει x++**. Γράφεις **x += 1** — και αντίστοιχα x -= 1.

// και %: τα δύο μισά της διαίρεσης

Όταν μοιράζεις 17 καραμέλες σε 5 παιδιά: πόσες παίρνει το καθένα και πόσες περισσεύουν;

```
>>> 17 // 5      # πόσες παίρνει το καθένα
3
>>> 17 % 5       # πόσες περισσεύουν
2
>>> 3 * 5 + 2     # επαλήθευση
17
```

Ο κανόνας που ισχύει πάντα

a = (a // b) * b + (a % b)

Το // λέει «πόσες ολόκληρες φορές χωράει», το % λέει «τι μένει».

ΠΟΎ ΧΡΗΣΙΜΟΠΟΙΟΎΝΤΑΙ

ΧΡΉΣΗ	ΚΏΔΙΚΑΣ → ΑΠΟΤΈΛΕΣΜΑ
Άρτιος / περιττός	`n % 2 → 0 ή 1`
Τελευταίο ψηφίο	`1987 % 10 → 7`
Χωρίς το τελευταίο	`1987 // 10 → 198`
Λεπτά σε ώρες	`135 // 60, 135 % 60 → 2, 15`

ΓΙΑ ΤΟ ΤΈΛΟΣ Αυτοί οι δύο τελεστές είναι η καρδιά της τελικής άσκησης «ρέστα σε κέρματα». Κράτα το παράδειγμα με τις καραμέλες.

Υπολόγισε με το χέρι

ΕΚΦΏΝΗΣΗ

Βρες το αποτέλεσμα κάθε έκφρασης **και τον τύπο του** (int ή float). Πρώτα στο χαρτί, μετά στο REPL.

ΛΎΣΗ

ΈΚΦΡΑΣΗ	ΑΠΟΤΈΛΕΣΜΑ	ΤΎΠΟΣ	ΣΧΌΛΙΟ
`2 + 3 * 4`	`14`	int	πρώτα ο πολλαπλασιασμός
`(2 + 3) * 4`	`20`	int	οι παρενθέσεις αλλάζουν σειρά
`10 / 2`	`5.0`	**float**	η / δίνει πάντα float
`23 // 4`	`5`	int	4 χωράει 5 φορές στο 23
`23 % 4`	`3`	int	23 − 20 = 3
`2 ** 10`	`1024`	int	δύναμη
`7 // 2.0`	`3.0`	**float**	ένα float → float αποτέλεσμα
`2 ** 3 ** 2`	`512`	int	**από δεξιά**: 2 ** 9

Ο γενικός κανόνας τύπου

Αν **έστω ένας** αριθμός είναι float, το αποτέλεσμα είναι float. Η **/** βγάζει float πάντα.

Η παγίδα

Το ** είναι ο μόνος τελεστής που υπολογίζεται **από δεξιά προς τα αριστερά**. 2 ** 3 ** 2 = 2 ** 9.

ΜΠΌΝΟΥΣ Δοκίμασε **-7 // 2** στο REPL. Βγάζει **-4** και όχι -3: η // στρογγυλεύει πάντα προς τα κάτω.

Ερωτήσεις με απάντηση True / False

ΤΕΛ.	ΣΗΜΑΊΝΕΙ	ΠΑΡΆΔΕΙΓΜΑ
==	ίσο	5 == 5.0 → True
!=	διάφορο	3 != 4 → True
< >	μικρότερο / μεγαλύτερο	2 > 7 → False
<= >=	ή ίσο	7 >= 7 → True

ΛΟΓΙΚΌΣ	TRUE ΌΤΑΝ
and	ισχύουν **και τα δύο**
or	ισχύει **έστω ένα**
not	**αντιστρέφει** την τιμή

```
grade = 15
print(grade >= 10 and grade <= 20)
print(10 <= grade <= 20)   # ίδιο!
True
True
```

Αλυσιδωτή σύγκριση

Η Python γράφει το «ανάμεσα» όπως στα μαθηματικά: **10 <= grade <= 20**. Ίδιο αποτέλεσμα με το and, πιο ευανάγνωστο.

ΜΗΝ ΜΠΕΡΔΕΎΕΙΣ = δίνει τιμή, == ρωτάει αν είναι ίσα. Στο if θα χρειαστείς πάντα το ==.

input(): ο χρήστης γράφει, εμείς διαβάζουμε

ΤΟ ΛΆΘΟΣ ΠΟΥ ΚΆΝΟΥΝ ΌΛΟΙ

```
a = input("Πρώτος: ")
b = input("Δεύτερος: ")
print(a + b)
Πρώτος: 5    Δεύτερος: 3
53
```

Η ΣΩΣΤΉ ΕΚΔΟΧΉ

```
a = int(input("Πρώτος: "))
b = int(input("Δεύτερος: "))
print(a + b)
Πρώτος: 5    Δεύτερος: 3
8
```

Γιατί 53;

Η **input() επιστρέφει πάντα str** — ακόμα κι αν ο χρήστης γράψει αριθμό. Και το + σε κείμενα τα **κολλάει**: "5" + "3" = "53".

Η λύση

Τυλίγουμε την input() σε **int()** ή **float()**. Διαβάζεται από μέσα προς τα έξω: πρώτα ρωτάει, μετά μετατρέπει.

Το μήνυμα μέσα στην input

Ό,τι γράψεις στις παρενθέσεις εμφανίζεται ως ερώτηση. Άφηνε ένα **κενό στο τέλος** ("Βάρος: ") για να μη κολλάει η απάντηση.

ΣΥΓΚΡΙΣΗ ΜΕ C Είναι το αντίστοιχο του **scanf**, χωρίς %d και & — αλλά τη μετατροπή την κάνεις εσύ.

int(), float(), str(), round()

ΚΛΉΣΗ	ΑΠΟΤΈΛΕΣΜΑ	ΤΙ ΣΥΝΈΒΗ
`int("42")`	`42`	κείμενο → ακέραιος
`float("1.75")`	`1.75`	κείμενο → δεκαδικός
`int(3.99)`	`3`	**κόβει**, δεν στρογγυλεύει
`round(3.99)`	`4`	στρογγυλεύει
`round(22.8571, 2)`	`22.86`	με 2 δεκαδικά
`str(2026)`	`"2026"`	αριθμός → κείμενο
`int("3.5")`	**ValueError**	το "3.5" δεν είναι ακέραιος
`int("πέντε")`	**ValueError**	δεν είναι αριθμός

int ή float στην input;

Αν η τιμή **μπορεί να έχει δεκαδικά** (βάρος, ύψος, θερμοκρασία, τιμή), διάβασέ τη με **float()**.

Τι γίνεται με το λάθος

Αν ο χρήστης γράψει «πέντε», το πρόγραμμα **σταματά** με ValueError. Πώς το προλαβαίνουμε: Μάθημα 5 (try/except).

ΧΡΥΣΌΣ ΚΑΝΌΝΑΣ Ό,τι μπορεί να έχει δεκαδικά, έστω και σπάνια, το διαβάζεις με **float()**. Ένας μέσος όρος δεν είναι ποτέ σίγουρα ακέραιος.

f-strings: κείμενο με τιμές μέσα

Βάζεις f πριν από τα εισαγωγικά και τις μεταβλητές μέσα σε { }.

```
name = "Νίκος"
bmi = 22.857142

print(f"Γεια σου {name}!")
print(f"ΔΜΣ: {bmi:.2f}")
Γεια σου Νίκος!
ΔΜΣ: 22.86
```

ΜΈΣΑ ΣΤΙΣ { }	ΑΠΟΤΈΛΕΣΜΑ	ΣΗΜΑΊΝΕΙ
{bmi}	22.857142	όπως είναι
{bmi:.2f}	22.86	2 δεκαδικά
{bmi:.0f}	23	χωρίς δεκαδικά
{7:02d}	07	2 ψηφία με 0

Γιατί το δείχνουμε από σήμερα

Στα προγράμματα του εργαστηρίου θέλουμε **2 δεκαδικά** και **ώρες σαν 02:03:04**. Αναλυτικά για τη μορφοποίηση: Μάθημα 8.

ΠΡΟΣΟΧΉ Το **:.2f** αλλάζει μόνο το πώς **εμφανίζεται** ο αριθμός. Η τιμή μέσα στη μεταβλητή bmi μένει ίδια.

Πολλαπλή εκχώρηση και +=

ΑΝΤΑΛΛΑΓΉ ΜΕ ΒΟΗΘΗΤΙΚΉ ΜΕΤΑΒΛΗΤΉ

```
temp = a
a = b
b = temp
```

ΑΝΤΑΛΛΑΓΉ ΜΕ ΠΟΛΛΑΠΛΉ ΕΚΧΏΡΗΣΗ

```
a, b = b, a
```

```
x, y = 3, 7        # x=3, y=7
i = j = 0          # και τα δύο 0

score = 10
score += 5         # 15
score *= 2         # 30
score //= 4        # 7
```

Τι κερδίζεις

a, b = b, a: η Python υπολογίζει πρώτα όλη τη δεξιά πλευρά και μετά μοιράζει τις τιμές. Δεν χρειάζεται temp.
Οι **+=**, **-=** και **//=** αλλάζουν τη μεταβλητή με βάση την τιμή της. Το **++** δεν υπάρχει.

ΧΡΉΣΙΜΟ ΣΤΑ ΡΈΣΤΑ Το **coins, rest = rest // 200, rest % 200** κάνει δύο υπολογισμούς σε μία γραμμή. Θα το δεις στη λύση.

Βρες το λάθος

ΕΚΦΏΝΗΣΗ

Κάθε κομμάτι έχει **ένα** λάθος — τα πιο συνηθισμένα των πρώτων εβδομάδων. Βρες το και διόρθωσέ το.

```
age = input("Ηλικία: ")
print(age + 1)
```

α TypeError: str + int → **int(input(...))**

```
count = 0
count++
```

β SyntaxError: δεν υπάρχει ++ → **count += 1**

```
x = 5
 print(x)
```

γ IndentationError: περιττό κενό → **σβήσε το**

```
if x = 5:
    print("πέντε")
```

δ SyntaxError: σύγκριση θέλει **==**

ΤΕΧΝΙΚΉ Διάβαζε το μήνυμα λάθους **από κάτω προς τα πάνω**: η τελευταία γραμμή λέει τι έγινε, η προηγούμενη πού.

Δείκτης Μάζας Σώματος

ΕΚΦΩΝΗΣΗ

Γράψε πρόγραμμα που διαβάζει **βάρος** (kg) και **ύψος** (m) και εμφανίζει τον ΔΜΣ με 2 δεκαδικά.
Τύπος: ΔΜΣ = βάρος / ύψος^2

ΑΝΑΜΕΝΟΜΕΝΗ ΈΞΟΔΟΣ

```
Βάρος σε kg: 70
Ύψος σε m: 1.75
ΔΜΣ: 22.86
```

ΛΎΣΗ

```python
# bmi.py — Δείκτης Μάζας Σώματος
weight = float(input("Βάρος σε kg: "))
height = float(input("Ύψος σε m: "))

bmi = weight / height ** 2

print(f"ΔΜΣ: {bmi:.2f}")
```

Γιατί float() και όχι int()

Το ύψος είναι **1.75** — με int("1.75") το πρόγραμμα θα έσκαγε με ValueError.

Γιατί χωρίς παρένθεση στο height ** 2

Η ** εκτελείται **πριν** από τη /. Το weight / (height ** 2) είναι σωστό αλλά περιττό.

ΕΠΈΚΤΑΣΗ Στο Μάθημα 2, με **if**, θα προσθέσουμε και την κατηγορία: λιποβαρής / φυσιολογικός / υπέρβαρος.

Μετατροπή θερμοκρασίας

ΕΚΦΏΝΗΣΗ

Διάβασε θερμοκρασία σε **Κελσίου** και εμφάνισέ τη σε **Φαρενάιτ** (1 δεκαδικό) και σε **Κέλβιν** (2 δεκαδικά). Τύποι: F = C × 9/5 + 32, K = C + 273.15

ΛΎΣΗ

```python
# temperature.py
c = float(input("Θερμοκρασία (°C): "))

f = c * 9 / 5 + 32
k = c + 273.15

print(f"{c}°C = {f:.1f}°F = {k:.2f}K")
```

ΕΊΣΟΔΟΣ °C	°F	K
0	32.0	273.15
25	77.0	298.15
100	212.0	373.15
-40	-40.0	233.15

Δοκίμασε τις τιμές του πίνακα

Ένα πρόγραμμα το ελέγχεις με **γνωστές απαντήσεις**: το νερό παγώνει στους 32°F. Στους **-40** οι δύο κλίμακες συμπίπτουν.

ΜΙΚΡΉ ΠΡΌΚΛΗΣΗ Γράψε και το αντίστροφο: από Φαρενάιτ σε Κελσίου. Λύσε τον τύπο ως προς C: **C = (F − 32) × 5/9**.

Ρέστα σε κέρματα

ΕΚΦΩΝΗΣΗ

Ο πελάτης χρωστά ένα ποσό και πληρώνει με χαρτονόμισμα. Το πρόγραμμα βρίσκει τα ρέστα και λέει **πόσα κέρματα από το καθένα** να δώσει ο ταμίας, με τα λιγότερα δυνατά κέρματα.
Κέρματα: 2€, 1€, 50λ, 20λ, 10λ, 5λ, 2λ, 1λ. Παράδειγμα: ποσό 15.12€, πληρωμή 20€ → ρέστα 4.88€.

ΛΎΣΗ

```
# change.py — Ρέστα σε κέρματα
price = float(input("Ποσό (€): "))
paid = float(input("Πληρωμή (€): "))

rest = round((paid - price) * 100)   # σε λεπτά
print(f"Ρέστα: {rest / 100:.2f}€")

c200, rest = rest // 200, rest % 200
c100, rest = rest // 100, rest % 100
c50, rest = rest // 50, rest % 50
c20, rest = rest // 20, rest % 20
c10, rest = rest // 10, rest % 10
c5, rest = rest // 5, rest % 5
c2, rest = rest // 2, rest % 2
c1 = rest

print("2€:", c200, " 1€:", c100, " 50λ:", c50, " 20λ:", c20)
print("10λ:", c10, " 5λ:", c5, " 2λ:", c2, " 1λ:", c1)
```

Η ιδέα

Για κάθε κέρμα, από το μεγαλύτερο: **//** λέει πόσα χωράνε, **%** λέει τι μένει για το επόμενο.

Γιατί δουλεύουμε σε λεπτά

Με ακέραιους δεν έχουμε σφάλματα δεκαδικών. Χωρίς **round()**: 0.29 × 100 = 28.9999… και το int() θα έδινε **28** αντί για 29.

ΔΟΚΊΜΑΣΕ Ποσό 0.71€, πληρωμή 1€ → ρέστα 0.29€ = 20λ + 5λ + 2λ + 2λ. Μετά βγάλε το round() και δες τι χαλάει.

Τι γίνεται για ρέστα 4.88€

Ακολουθούμε τη μεταβλητή rest γραμμή προς γραμμή. Ξεκινά από 488 λεπτά.

ΚΈΡΜΑ	REST ΠΡΙΝ	REST // ΚΈΡΜΑ	REST % ΚΈΡΜΑ (ΜΈΝΕΙ)
2€ (200)	488	**2**	88
1€ (100)	88	**0**	88
50λ	88	**1**	38
20λ	38	**1**	18
10λ	18	**1**	8
5λ	8	**1**	3
2λ	3	**1**	1
1λ	1	**1**	0

```
Ποσό (€): 15.12
Πληρωμή (€): 20
Ρέστα: 4.88€
2€: 2  1€: 0  50λ: 1  20λ: 1
10λ: 1  5λ: 1  2λ: 1  1λ: 1
```

Επαλήθευση

2·200 + 50 + 20 + 10 + 5 + 2 + 1 = **488** ✓ Σύνολο **8 κέρματα**.

ΤΕΧΝΙΚΉ Ο πίνακας τιμών είναι ο πιο γρήγορος τρόπος να βρεις γιατί ένα πρόγραμμα βγάζει λάθος.

Δευτερόλεπτα σε ώρες:λεπτά:δευτ.

ΕΚΦΏΝΗΣΗ

Διάβασε ένα πλήθος δευτερολέπτων και εμφάνισέ το σε μορφή **ΩΩ:ΛΛ:ΔΔ**. Παράδειγμα: 7384 → **02:03:04**. Ίδια ιδέα με τα ρέστα.

ΛΎΣΗ

```
# clock.py
total = int(input("Δευτερόλεπτα: "))

h, rest = total // 3600, total % 3600
m, s = rest // 60, rest % 60

print(f"{h:02d}:{m:02d}:{s:02d}")
02:03:04
```

ΒΉΜΑ	ΠΡΆΞΗ	ΑΠΟΤΈΛΕΣΜΑ
ώρες	`7384 // 3600`	2
μένουν	`7384 % 3600`	184
λεπτά	`184 // 60`	3
δευτερόλεπτα	`184 % 60`	4

Το :02d

Γεμίζει με **μηδενικό** μπροστά ώστε να έχει 2 ψηφία: 3 → **03**. Χωρίς αυτό θα έβγαινε 2:3:4.

ΠΑΡΑΤΉΡΗΣΗ Ρέστα, ώρες, ψηφία αριθμού: το ίδιο μοτίβο // και % λύνει όλα τα προβλήματα «σπάσε σε μονάδες».

Τι κρατάμε

Η σύνταξη

Εσοχές αντί για αγκύλες. **Καμία δήλωση** τύπου. Κανένα **;** και κανένα **++**.

input() = str

Ό,τι διαβάζεις είναι **κείμενο**. Μετατροπή με **int()** ή **float()** πριν από πράξεις.

/ // %

/ δίνει πάντα float. **//** ακέραιο πηλίκο. **%** υπόλοιπο. Μαζί σπάνε ποσότητες σε μονάδες.

Σήμερα έγραψες τρία πλήρη προγράμματα: **ΔΜΣ**, **θερμοκρασία** και **ρέστα σε κέρματα**. Όλα έχουν την ίδια δομή: **διάβασε → υπολόγισε → εμφάνισε**.

Επόμενο μάθημα · Δευ 12/10

Δομές επιλογής και επανάληψης: **if / elif / else**, **while**, **for**. Ο ΔΜΣ θα αποκτήσει κατηγορία και τα ρέστα θα ελέγχουν αν η πληρωμή φτάνει.

ΓΙΑ ΤΟ ΣΠΊΤΙ Πρόγραμμα που διαβάζει τριψήφιο αριθμό και εμφανίζει το **άθροισμα των ψηφίων** του (π.χ. 472 → 13). Μόνο με // και %.